

GPU programming models and languages

GPU programming in computational engineering

Escuela de Doctorado 2025/26

University of Zaragoza

Sergio Martínez-Aranda

sermar@unizar.es

Mario Morales-Hernández

mmorales@unizar.es

Fluid Dynamics Technologies - I3A, University of Zaragoza



Universidad
Zaragoza



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza



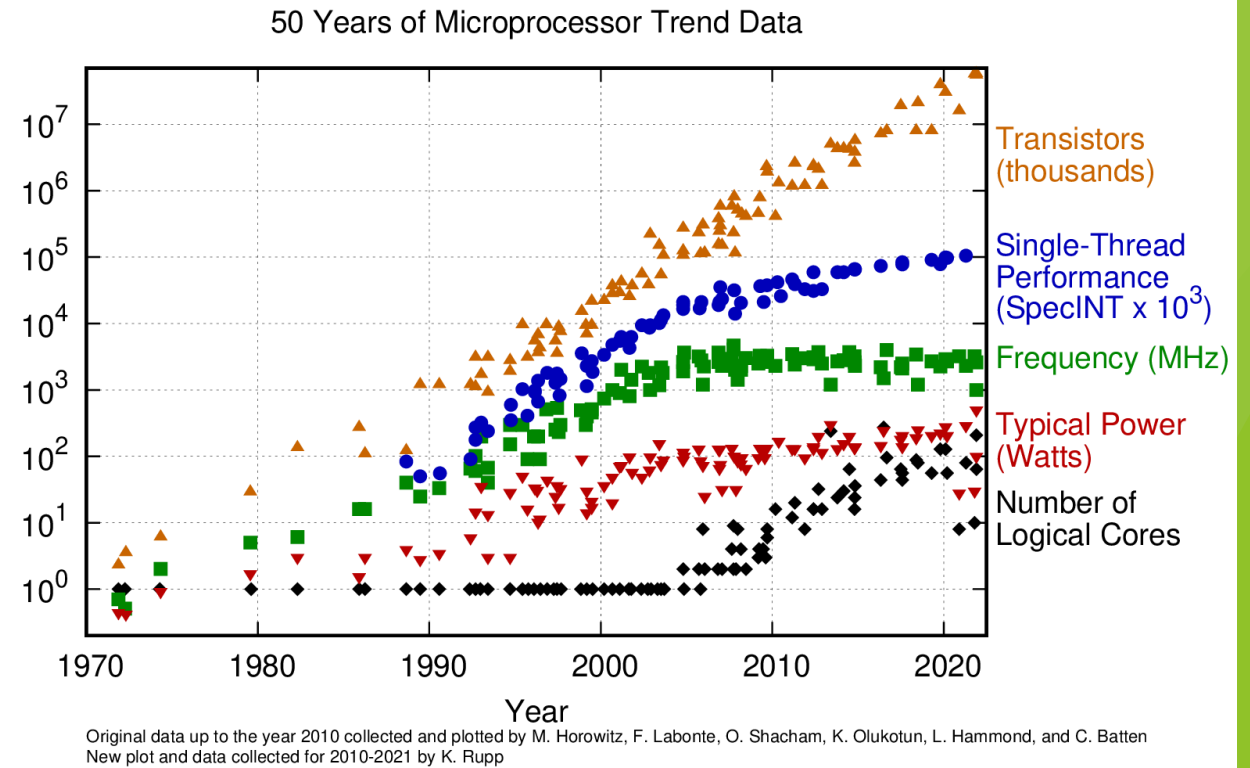
Instituto Universitario de Investigación
en Ingeniería de Aragón
Universidad Zaragoza

Outline

- ▶ Introduction
- ▶ Programming models and languages
 - CUDA/SYCL
 - OpenMP/OpenACC
 - Kokkos
- ▶ Memory, abstraction and choice

Motivation: Why embrace GPU computing?

- Beyond Moore's law
- Unprecedented speed and efficiency
- Expanding frontiers of research and development
- Cost-effective scalability
- Versatility across fields
- Future-proofing projects



Moving to GPU

**"I have my code in CPU (C, C++, Fortran, Python).
How can I run it on GPUs?"**

Moving to GPU

Step 1 | Assessment

Identify portions of the code that could benefit from parallelization on a GPU



Step 2 | Porting/Adapting

Choose a suitable GPU programming model/language



Step 3 | Parallel algorithm design

Redesign algorithms to exploit the parallel processing capabilities of GPUs: rethinking data structures, memory access patterns...

Step 5 | Testing and validation

Ensure correctness to verify that no errors were introduced



Step 4 | Implementation

Translate the code to the chosen programming model



Moving to GPU

**"Considering the effort to port code to GPUs,
does it pay off in terms of performance gains?"**

Moving to GPU

Performance gain ?

Significant speedups possible but depends on problem nature.

Effort consideration ?

Initial porting and optimization can be intricate and time consuming: redesign algorithms, optimizing memory access and tuning performance. Also, validation and verification.

Trade-offs ?

Not all code is suited for GPUs. And it requires and investment in learning and development.

Bigger picture ?

Future-proofing and scalability.
Expanding computational horizons.

GPU acceleration is a technical decision, but also a software strategy.

Programming models and languages

► Diverse ecosystem with multiple vendors and languages

Native

- CUDA (NVIDIA)
- HIP (AMD)
- SYCL (Intel)

Directive-based

- OpenACC
- OpenMP

Higher-level abstraction

- Kokkos
- RAJA
- ALPAKA

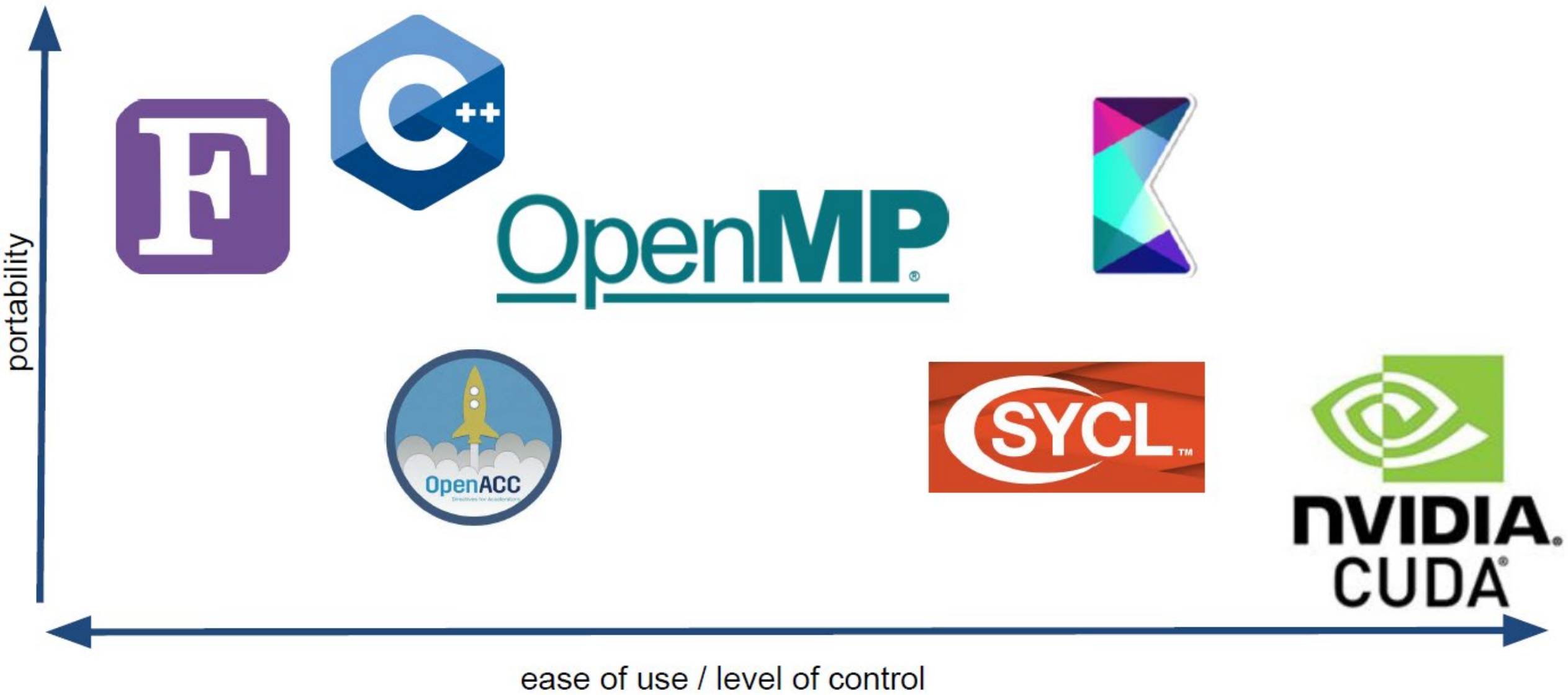
- Full vendor support
- ◐ Indirect, but comprehensive support, by vendor
- ◑ Vendor support, but not (yet) entirely comprehensive

- ▲ Comprehensive support, but not by vendor
- ★ Limited, probably indirect support – but at least some
- ✍ No direct support available

C++ C++ (sometimes also C)
Fortran Fortran

	CUDA		HIP		SYCL		OpenACC		OpenMP		Standard		Kokkos		ALPAKA		Python
	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	
NVIDIA	● 1	● 2	◐ 3	★ 4	▲ 5	✍ 6	● 7	● 8	◑ 9	● 10	● 11	● 12	▲ 13	★ 14	▲ 15	✍ 16	● 17
AMD	◐ 18	★ 19	● 20	★ 4	▲ 21	✍ 6	▲ 22	▲ 23	● 24	● 25	▲ 26	✍ 27	▲ 28	★ 14	▲ 29	✍ 16	★ 30
Intel	◐ 31	✍ 32	▲ 33	✍ 34	● 35	✍ 6	★ 36	★ 37	● 38	● 39	● 40	● 41	▲ 42	★ 14	▲ 43	✍ 16	◑ 44

Herten, A. (2023). Many Cores Many Models: GPU Programming Model vs. Vendor Compatibility Overview. arXiv:2309.05445v3 [cs.DC]. Retrieved from <https://arxiv.org/abs/2309.05445>



Programming models and languages

► Diverse ecosystem with multiple vendors and languages

Native

- CUDA (NVIDIA)
- HIP (AMD)
- SYCL (Intel)

1

Directive-based

- OpenACC
- OpenMP

Higher-level abstraction

- Kokkos
- RAJA
- ALPAKA

- Full vendor support
- ◐ Indirect, but comprehensive support, by vendor
- ◑ Vendor support, but not (yet) entirely comprehensive

- ▲ Comprehensive support, but not by vendor
- ★ Limited, probably indirect support – but at least some
- ✍ No direct support available

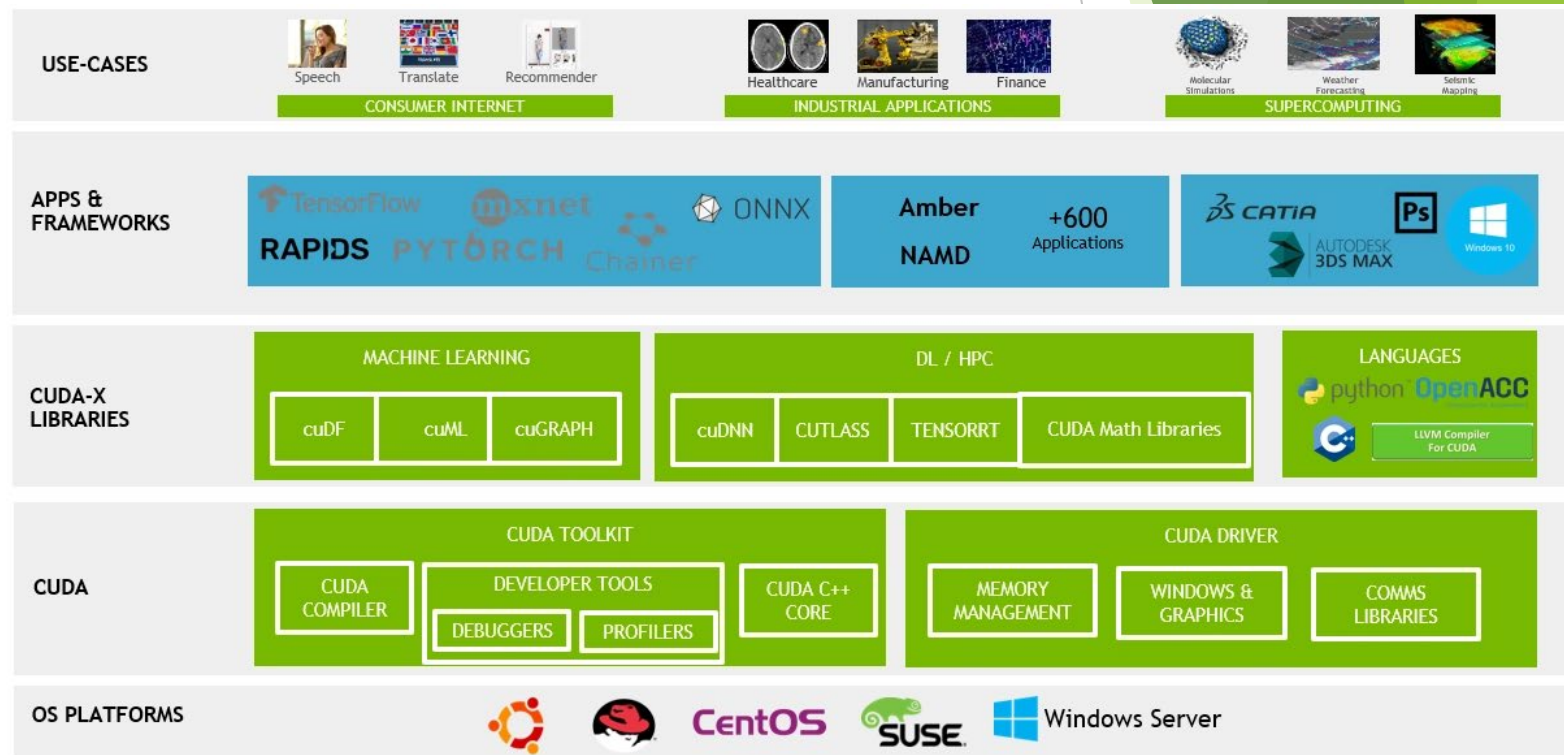
C++ C++ (sometimes also C)
Fortran Fortran

	CUDA		HIP		SYCL		OpenACC		OpenMP		Standard		Kokkos		ALPAKA		Python
	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	
NVIDIA	● 1	● 2	◐ 3	★ 4	▲ 5	✍ 6	● 7	● 8	◑ 9	◑ 10	● 11	● 12	▲ 13	★ 14	▲ 15	✍ 16	● 17
AMD	◐ 18	★ 19	● 20	★ 4	▲ 21	✍ 6	▲ 22	▲ 23	● 24	● 25	▲ 26	✍ 27	▲ 28	★ 14	▲ 29	✍ 16	★ 30
Intel	◐ 31	✍ 32	▲ 33	✍ 34	● 35	✍ 6	★ 36	★ 37	● 38	● 39	● 40	● 41	▲ 42	★ 14	▲ 43	✍ 16	◑ 44

Herten, A. (2023). Many Cores Many Models: GPU Programming Model vs. Vendor Compatibility Overview. arXiv:2309.05445v3 [cs.DC]. Retrieved from <https://arxiv.org/abs/2309.05445>

What is CUDA

- ▶ CUDA (Compute Unified Device Architecture) is a parallel computing platform and programming model created by NVIDIA in 2006.
- ▶ It includes a comprehensive development toolkit with compilers, libraries, and debugging tools designed to assist in developing software that runs on NVIDIA GPUs.
- ▶ Support for various programming languages (C, C++, Fortran, Python...)
- ▶ Wide-ranging use cases: HPC, deep learning, big data ...



CUDA: Implementation of loops

Standard C Code

```
void saxpy(int n, float a,
          float *x, float *y)
{
    for (int i = 0; i < n; ++i)
        y[i] = a*x[i] + y[i];
}

int N = 1<<20;

// Perform SAXPY on 1M elements
saxpy(N, 2.0, x, y);
```

C with CUDA extensions

```
__global__
void saxpy(int n, float a,
          float *x, float *y)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i < n) y[i] = a*x[i] + y[i];
}

int N = 1<<20;
cudaMemcpy(x, d_x, N, cudaMemcpyHostToDevice);
cudaMemcpy(y, d_y, N, cudaMemcpyHostToDevice);

// Perform SAXPY on 1M elements
saxpy<<<4096,256>>>(N, 2.0, x, y);

cudaMemcpy(d_y, y, N, cudaMemcpyDeviceToHost);
```

CUDA: Implementation of loops

Standard C Code

```
void saxpy(int n, float a,
          float *x, float *y)
{
    for (int i = 0; i < n; ++i)
        y[i] = a*x[i] + y[i];
}

int N = 1<<20;

// Perform SAXPY on 1M elements
saxpy(N, 2.0, x, y);
```

C with CUDA extensions

```
__global__
void saxpy(int n, float a,
          float *x, float *y)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i < n) y[i] = a*x[i] + y[i];
}

int N = 1<<20;
cudaMemcpy(x, d_x, N, cudaMemcpyHostToDevice);
cudaMemcpy(y, d_y, N, cudaMemcpyHostToDevice);

// Perform SAXPY on 1M elements
saxpy<<<4096,256>>>(N, 2.0, x, y);

cudaMemcpy(d_y, y, N, cudaMemcpyDeviceToHost);
```

- ▶ for
i=1..nElements
- ▶ A[i]=...
- ▶ end for

CUDA: Implementation of loops

Standard C Code

```
void saxpy(int n, float a,
          float *x, float *y)
{
    for (int i = 0; i < n; ++i)
        y[i] = a*x[i] + y[i];
}

int N = 1<<20;

// Perform SAXPY on 1M elements
saxpy(N, 2.0, x, y);
```

C with CUDA extensions

```
__global__
void saxpy(int n, float a,
          float *x, float *y)
{
    int i = blockIdx.x*blockDim.x + threadIdx.x;
    if (i < n) y[i] = a*x[i] + y[i];
}

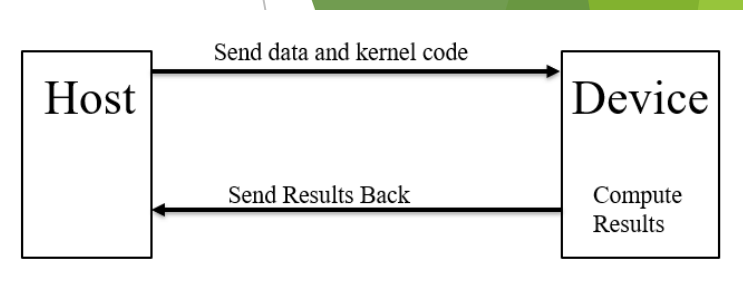
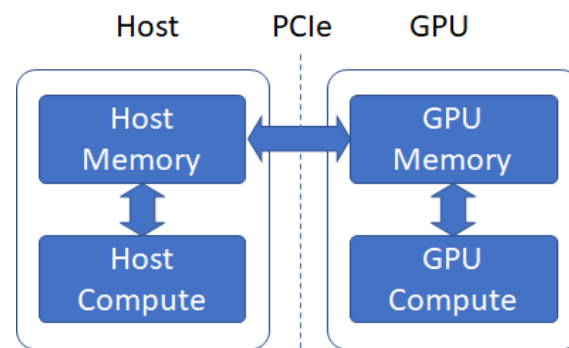
int N = 1<<20;
cudaMemcpy(x, d_x, N, cudaMemcpyHostToDevice);
cudaMemcpy(y, d_y, N, cudaMemcpyHostToDevice);

// Perform SAXPY on 1M elements
saxpy<<<4096,256>>>(N, 2.0, x, y);

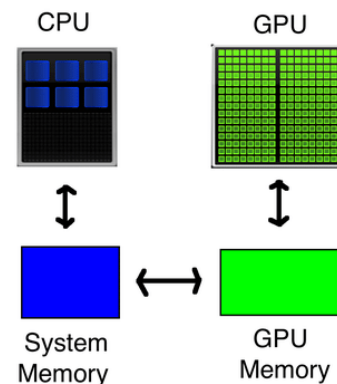
cudaMemcpy(d_y, y, N, cudaMemcpyDeviceToHost);
```

CUDA: Implementation of loops

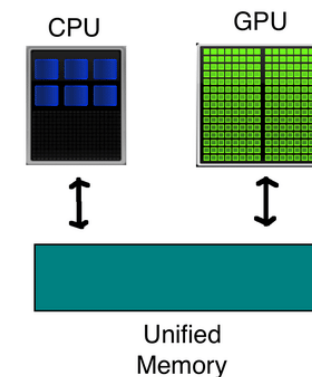
- **Dynamic Allocation:** Use `cudaMalloc()` for creating space in GPU memory and `malloc()` for CPU memory before data transfer.
- **Memory Copy:** `cudaMemcpy()` transfers data between CPU and GPU. Opt for `cudaMemcpyAsync()` for non-blocking transfers.
- **Unified Memory:** `cudaMallocManaged()` allows data to be automatically shared between CPU and GPU, simplifying code but watch for performance.
- **Key Tips:**
 - Opt for Unified Memory for simplicity; direct allocation and transfer for performance.
 - Minimize data transfers for speed; use asynchronous operations to overlap computation and transfer.



View without 'Unified Memory'



View with 'Unified Memory'



Programming models and languages

► Diverse ecosystem with multiple vendors and languages

Native

- CUDA (NVIDIA)
- HIP (AMD)
- SYCL (Intel)

1

Directive-based

- OpenACC
- OpenMP

Higher-level abstraction

- Kokkos
- RAJA
- ALPAKA

- Full vendor support
- ◐ Indirect, but comprehensive support, by vendor
- ◑ Vendor support, but not (yet) entirely comprehensive

- ▲ Comprehensive support, but not by vendor
- ★ Limited, probably indirect support – but at least some
- ✍ No direct support available

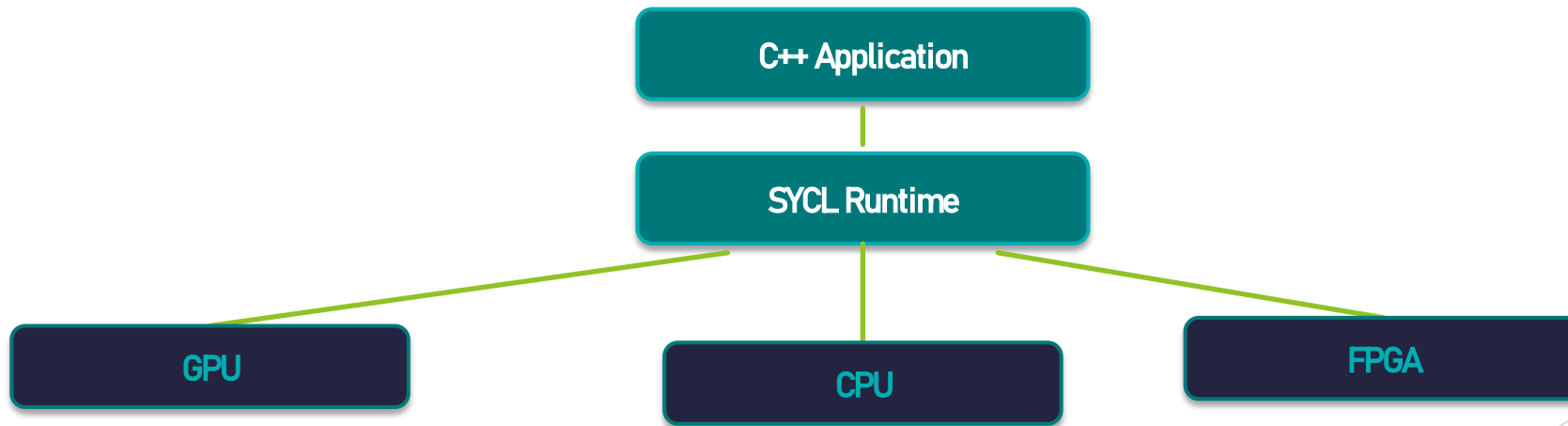
C++ C++ (sometimes also C)
Fortran Fortran

	CUDA		HIP		SYCL		OpenACC		OpenMP		Standard		Kokkos		ALPAKA		Python
	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	
NVIDIA	● 1	● 2	◐ 3	★ 4	▲ 5	✍ 6	● 7	● 8	◑ 9	◑ 10	● 11	● 12	▲ 13	★ 14	▲ 15	✍ 16	● 17
AMD	◐ 18	★ 19	● 20	★ 4	▲ 21	✍ 6	▲ 22	★ 23	● 24	● 25	▲ 26	✍ 27	▲ 28	★ 14	▲ 29	✍ 16	★ 30
Intel	◐ 31	✍ 32	▲ 33	✍ 34	● 35	✍ 6	★ 36	★ 37	● 38	● 39	● 40	● 41	▲ 42	★ 14	▲ 43	✍ 16	◑ 44

Herten, A. (2023). Many Cores Many Models: GPU Programming Model vs. Vendor Compatibility Overview. arXiv:2309.05445v3 [cs.DC]. Retrieved from <https://arxiv.org/abs/2309.05445>

Introduction to SYCL

- SYCL: industry-standard, single-source C++ programming model from the Khronos Group
- Write standard C++ that executes on heterogeneous devices (GPU, CPU, FPGA)
- No new language → pure C++ templates and lambda functions
- Key implementations: Intel oneAPI DPC++, AdaptiveCpp (formerly hipSYCL), triSYCL
- Integral part of Intel oneAPI: unified, cross-architecture programming
- SYCL 2020 simplifies CUDA migration with USM and sub-groups



SYCL brings CUDA-level performance to industry-standard C++ — no vendor lock-in, no custom language extensions.

SYCL Execution Model: Queue, Device, parallel_for

```
#include <sycl/sycl.hpp>

int main() {
    sycl::queue q(sycl::gpu_selector_v);

    // Allocate USM device memory
    double *a = sycl::malloc_device<double>(N, q);
    double *b = sycl::malloc_device<double>(N, q);
    double *c = sycl::malloc_device<double>(N, q);

    q.parallel_for(N, [=](sycl::id<1> i) {
        c[i] = a[i] + b[i];
    }).wait();

    sycl::free(a, q);
    return 0;
}
```

queue: Submission channel to a device — commands execute asynchronously

device: Selected via selector (gpu_selector_v, cpu_selector_v, default_selector_v)

parallel_for: Enqueues a kernel — replaces CUDA <<<grid, block>>> syntax

All work is expressed through lambda functions capturing C++ variables

Submit work to a queue → the runtime dispatches to the device. Asynchronous by default, explicit when needed.

SYCL Memory Model: Buffers vs. Unified Shared Memory

Buffer / Accessor Model

```
buffer<T, 1> buf(data, range<1>(N));  
auto acc =  
buf.get_access<access::mode::read_write>(cgh);
```

Data managed by the SYCL runtime

Runtime tracks dependencies automatically

Safer, higher-level — but more verbose

Implicit — runtime-managed

Unified Shared Memory (USM)

```
double *a = malloc_device<double>(N, q);  
double *s = malloc_shared<double>(N, q);
```

Explicit pointer-based allocation

Programmer controls data movement (like CUDA)

Lower-level, easier CUDA migration path

Explicit — programmer-managed

Buffers = safety + automation; USM = control + CUDA-familiarity. Choose based on your performance tuning needs.

Programming models and languages

► Diverse ecosystem with multiple vendors and languages

Native

- CUDA (NVIDIA)
- HIP (AMD)

Directive-based

- OpenACC
- OpenMP

2

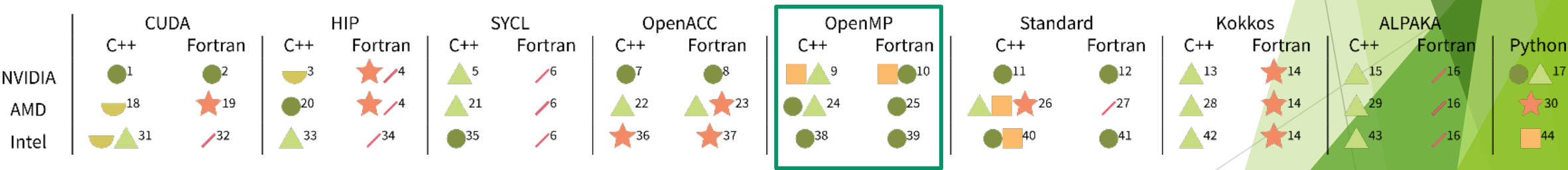
Higher-level abstraction

- Kokkos
- RAJA
- ALPAKA

- Full vendor support
- ◐ Indirect, but comprehensive support, by vendor
- ◑ Vendor support, but not (yet) entirely comprehensive

- ▲ Comprehensive support, but not by vendor
- ★ Limited, probably indirect support – but at least some
- ／ No direct support available

C++ C++ (sometimes also C)
Fortran Fortran



Herten, A. (2023). Many Cores Many Models: GPU Programming Model vs. Vendor Compatibility Overview. arXiv:2309.05445v3 [cs.DC]. Retrieved from <https://arxiv.org/abs/2309.05445>

Evolution of OpenMP

OpenMP 1.0 (1997): Directive-based shared-memory parallelism for multi-core CPUs

OpenMP 3.0 (2008): Tasking model for irregular parallelism

OpenMP 4.0 (2013): target constructs for GPU/offload acceleration

OpenMP 5.0 (2018): Unified memory, omp target refinements, reverse offload

OpenMP 5.2 (2021): Improved interoperability, device-aware constructs



OpenMP evolved from CPU-only loops to heterogeneous GPU offloading — one API, multiple architectures.

OpenMP: Classical CPU Parallelism

```
#include <omp.h>
#include <vector>

int main() {
    std::vector<double> a(1000000), b(1000000), c(1000000);
    // Initialize a and b...

    #pragma omp parallel for
    for (int i = 0; i < 1000000; i++) {
        c[i] = a[i] + b[i];
    }
    return 0;
}
```

#pragma omp parallel for distributes loop iterations across CPU threads

Fork-join model: master thread forks a team of threads, joins at end

Data-sharing: variables outside the loop are shared by default (use private(), reduction() to control)



A single #pragma transforms a serial loop into multi-threaded code — minimal effort, immediate CPU speedup.

OpenMP Target Offload for GPUs

```
#pragma omp target teams distribute parallel for \  
    map(to: a[0:N], b[0:N]) map(from: c[0:N])  
for (int i = 0; i < N; i++) {  
    c[i] = a[i] + b[i];  
}
```

target: Offload execution to a device (GPU)

teams: Create teams of thread blocks on the device

distribute: Distribute loop iterations across teams

parallel for: Each team runs iterations in parallel

map(to:...) / map(from:...): Control host  device data movement



One directive chain replaces CUDA kernel launches + memory copies — portability with single-source code.

Programming models and languages

► Diverse ecosystem with multiple vendors and languages

Native

- CUDA (NVIDIA)
- HIP (AMD)
- SYCL (Intel)

Directive-based

- OpenACC
- OpenMP

2

Higher-level abstraction

- Kokkos
- RAJA
- ALPAKA

- Full vendor support
- ◐ Indirect, but comprehensive support, by vendor
- ◑ Vendor support, but not (yet) entirely comprehensive

- ▲ Comprehensive support, but not by vendor
- ★ Limited, probably indirect support – but at least some
- ✍ No direct support available

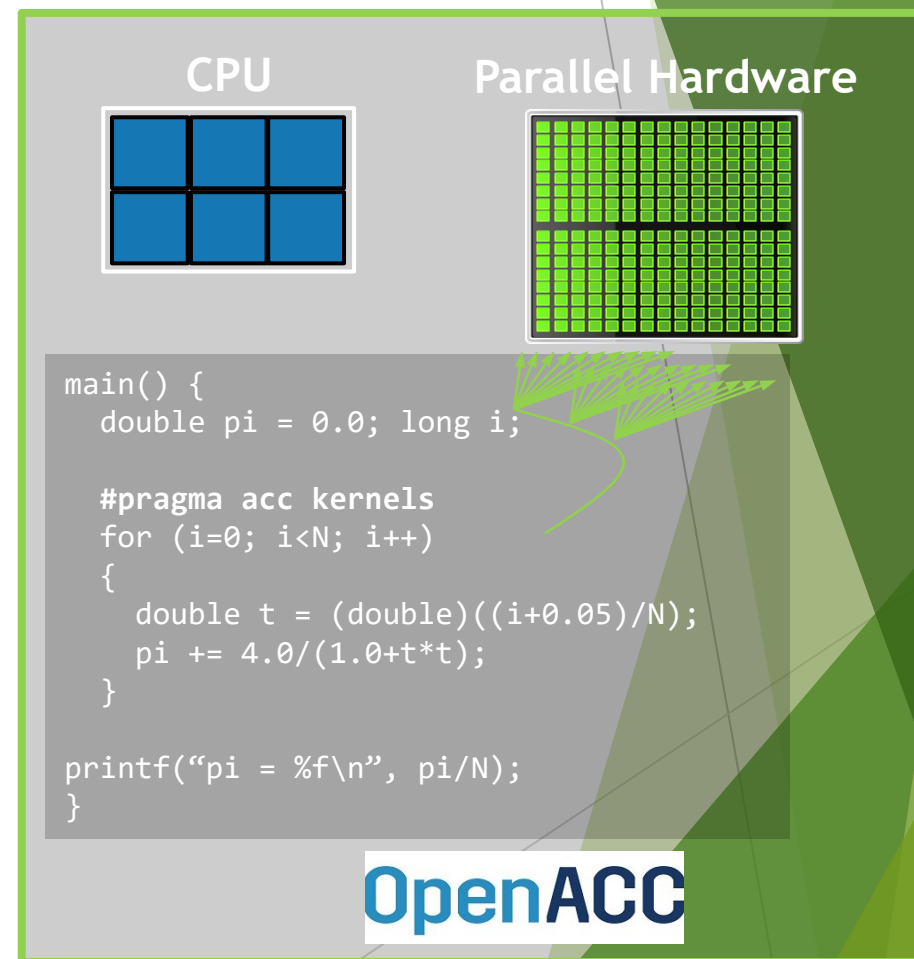
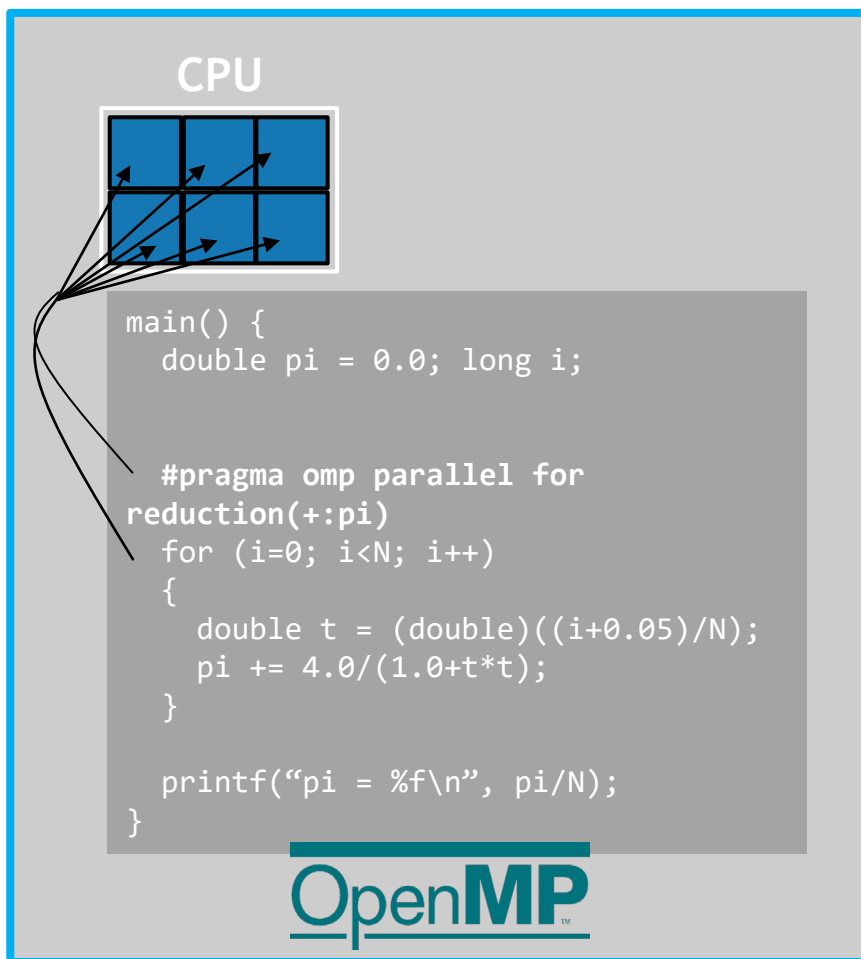
C++ C++ (sometimes also C)
Fortran Fortran

	CUDA		HIP		SYCL		OpenACC		OpenMP		Standard		Kokkos		ALPAKA		Python
	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	C++	Fortran	
NVIDIA	●1	●2	◐3	★4	▲5	✍6	●7	●8	◑9	◑10	●11	●12	▲13	★14	▲15	✍16	●17
AMD	◐18	★19	●20	★4	▲21	✍6	▲22	★23	●24	●25	▲26	✍27	▲28	★14	▲29	✍16	★30
Intel	◐31	✍32	▲33	✍34	●35	✍6	★36	★37	●38	●39	●40	●41	▲42	★14	▲43	✍16	◑44

Herten, A. (2023). Many Cores Many Models: GPU Programming Model vs. Vendor Compatibility Overview. arXiv:2309.05445v3 [cs.DC]. Retrieved from <https://arxiv.org/abs/2309.05445>

OpenACC: Familiar to OpenMP programmers

- A directive-based approach to easily accelerate code on GPUs and CPUs with minimal changes.



What is OpenACC

- ▶ A directive-based approach to easily accelerate code on GPUs and CPUs with minimal changes.
- ▶ **Simple to Use:** Just add directives to code sections you want to run in parallel—no need for detailed GPU programming knowledge
- ▶ **Works on Various Devices:** Designed for multi-platform compatibility, including NVIDIA and AMD GPUs.
- ▶ **Gradual Optimization:** Start optimizing critical code sections first for noticeable performance boosts.
- ▶ **Compiler Friendly:** Supported by multiple compilers for straightforward code compilation and optimization.
- ▶ Ideal for developers seeking performance gains without deep diving into complex GPU code
- ▶ For Fortran users, OpenACC is a particularly good option

Add Simple Compiler Directive

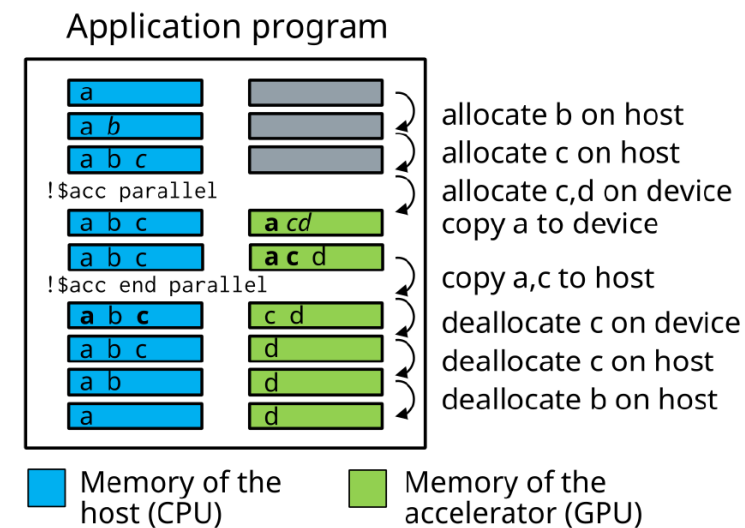
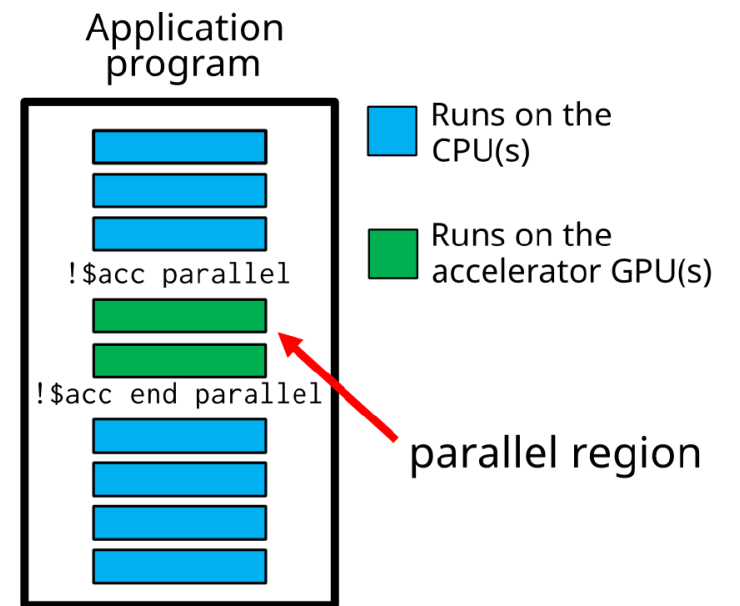
```
main()
{
    <serial code>
    #pragma acc kernels
    {
        <parallel code>
    }
}
```



OpenACC execution model

- 1 Program runs on the host CPU
- 2 Host offloads compute-intensive regions (kernels) and related data to the accelerator GPU
- 3 Compute kernels are executed by the GPU

OpenACC exposes the separate memories through data environment that defines the memory management and needed copy operations



OpenACC: compute directives and data environment

- ▶ OpenACC uses **compiler directives** for defining **compute regions** (and data transfers) that are to be performed on a GPU
- ▶ **Important constructs**: parallel, kernels, data, loop, update, host_data, wait
- ▶ Often **used clauses**: if (condition), async(handle)

	sentinel	construct	clauses
C/C++	#pragma acc	kernels	copy(data)
Fortran	!\$acc	kernels	copy(data)

It supports devices which either share memory with or have a separate memory from the host

Constructs and clauses for:

- defining the variables on the device
- transferring data to/from the device

C/C++

```
int a[100], d[3][3], i, j;

#pragma acc data copy(a[0:100])
{
    #pragma acc parallel loop present(a)
    for (int i=0; i<100; i++)
        a[i] = a[i] + 1;
    #pragma acc parallel loop \
        collapse(2) copyout(d)
    for (int i=0; i<3; ++i)
        for (int j=0; j<3; ++j)
            d[i][j] = i*3 + j + 1;
}
```

Fortran

```
integer a(0:99), d(3,3), i, j

!$acc data copy(a(0:99))
!$acc parallel loop present(a)
do i=0,99
    a(i) = a(i) + 1
end do
!$acc end parallel loop
!$acc parallel loop collapse(2) copyout(d)
do j=1,3
    do i=1,3
        d(i,j) = i*3 + j + 1
    end do
end do
!$acc end parallel loop
!$acc end data
```

OpenMP vs. OpenACC: A Practical Comparison

Aspect	OpenMP	OpenACC
Origin	ARB (Architecture Review Board)	OpenACC Consortium (NVIDIA, PGI, Cray)
Style	Structured blocks + directives	Descriptive, compiler-driven directives
GPU Support	Since v4.0 (2013), target constructs	Designed for GPUs from day one (2011)
Portability	CPU + GPU + FPGA via same directives	Primarily GPU-focused
Ecosystem	GCC, Clang, Intel, NVIDIA HPC	NVIDIA HPC SDK, GCC (partial)
Learning Curve	Moderate (rich feature set)	Lower (fewer constructs, simpler)

*OpenMP = broad heterogeneous scope; OpenACC = GPU-first philosophy. Choose based on your hardware mix. Both: **Incremental porting!***

OpenACC vs. CUDA/HIP

Why OpenACC and not CUDA/HIP?

- Easier to work with
- Porting of existing software requires less work
- Same code can be compiled to CPU and GPU versions easily

Why CUDA/HIP and not OpenACC?

- Can access all features of the GPU hardware
- More optimization possibilities

Programming models and languages

► Diverse ecosystem with multiple vendors and languages

Native

- CUDA (NVIDIA)
- HIP (AMD)
- SYCL (Intel)

Directive-based

- OpenACC
- OpenMP

Higher-level abstraction

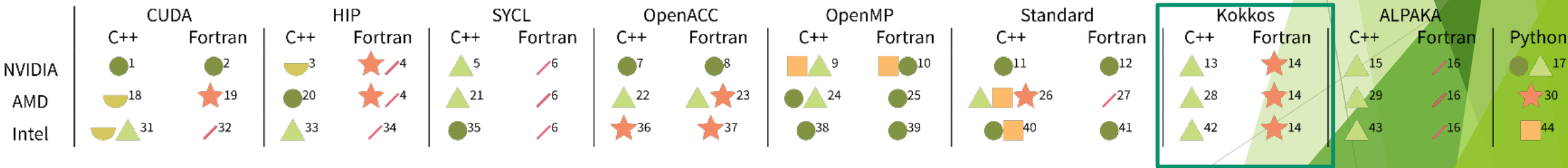
- Kokkos
- RAJA
- ALPAKA

3

- Full vendor support
- ◐ Indirect, but comprehensive support, by vendor
- ◑ Vendor support, but not (yet) entirely comprehensive

- ▲ Comprehensive support, but not by vendor
- ★ Limited, probably indirect support – but at least some
- ／ No direct support available

C++ C++ (sometimes also C)
Fortran Fortran



Herten, A. (2023). Many Cores Many Models: GPU Programming Model vs. Vendor Compatibility Overview. arXiv:2309.05445v3 [cs.DC]. Retrieved from <https://arxiv.org/abs/2309.05445>

What is Kokkos

- A programming model for performance portability

Performance Portability

“Achieving a consistent ratio of the actual time to solution to either the best-known or the theoretical best time to solution on each platform with minimal platform specific code required.”

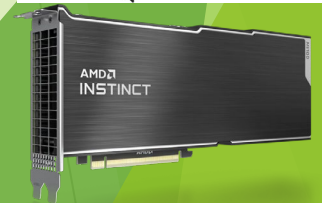
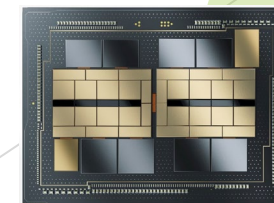
Source: <https://performanceportability.org/>

Future-proofing

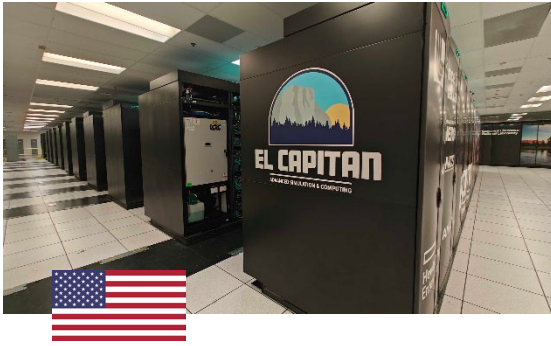
“To design software, a computer, etc. so that it can still be used in the future, even when technology changes.”

Source: Cambridge Dictionary

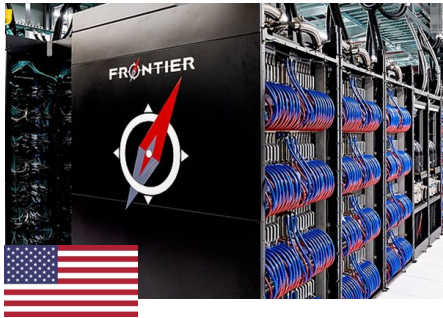
- Lifetime of software is much longer than lifetime of hardware
- Hardware technology changes, nowadays into a variety of new solutions
- For scientists and developers the code needs to be readable, maintainable, and compact, avoiding code duplications and hiding technicalities (separation of concerns)
- Use your code on CPUs and GPUs (and more), from laptops to workstations, clusters and HPC systems.
- Avoid re-coding and multiple hardware-specific code bases.



Exascale and performance-portability



El Capitán@LLNL (USAn)
CPU: AMD Epyc
GPU: AMD Instinct
Perf. peak: 1,74 Eflops
Deployment: 2024
#1 in TOP500



Frontier@ORNL (USA)
CPU: AMD EPYC
GPU: AMD Instinct
Perf. peak: 1.3 Eflops
Deployment: 2022
#2 in TOP500



Aurora@Argonne (USA)
CPU: Intel Xeon Sapphire Rapids
GPU: Intel Xe
Perf. peak: 1,01 Eflops
Deployment: 2023
#3 in TOP500



Fugaku @ Reiken (Japan)
CPU: ARM
GPU: none
Perf. Peak: 0.44 Eflops
Deployment: 2021
#6 in TOP500



Lumi Consortium (Finland)
CPU: AMD EPYC
GPU: AMD Instinct
Perf peak: 0.38 Eflops
Deployment: 2021/2022
#8 in TOP500



Leonardo@CINECA (Italy)
CPU: Intel Xeon Ice Lake & Intel Xeon Sapphire
GPU: Nvidia A100
Perf. peak: 0.24 Eflops
Deployment: 2022
#9 in TOP500

November 2024: 1/10 no GPUs, 3/10 Nvidia GPUs, 5/10 AMD GPUs, 1/10 Intel GPUs
November 2023: 1/10 no GPUs, 6/10 Nvidia GPUs, 2/10 AMD GPUs, 1/10 Intel GPUs
November 2022: 3/10 no GPUs, 5/10 Nvidia GPUs, 2/10 AMD GPUs, 0/10 Intel GPUs
November 2021: 3/10 no GPUs, 7/10 Nvidia GPUs, 0/10 AMD GPUs, 0/10 Intel GPUs

What is Kokkos

► Ensuring performance portability and future proofing

Open Source C++ library

Developed at Sandia National Lab (USA).

Community project

Programming model that targets all major HPC platforms

Avoid the cost of porting codes

It combines data structures with parallel execution

- Memory spaces and layouts
- Execution patterns (for, reduction, atomic...)
- Data allocation, transfer and execution spaces

Math libraries and tools based on Kokkos

- Profiling
- Debugging

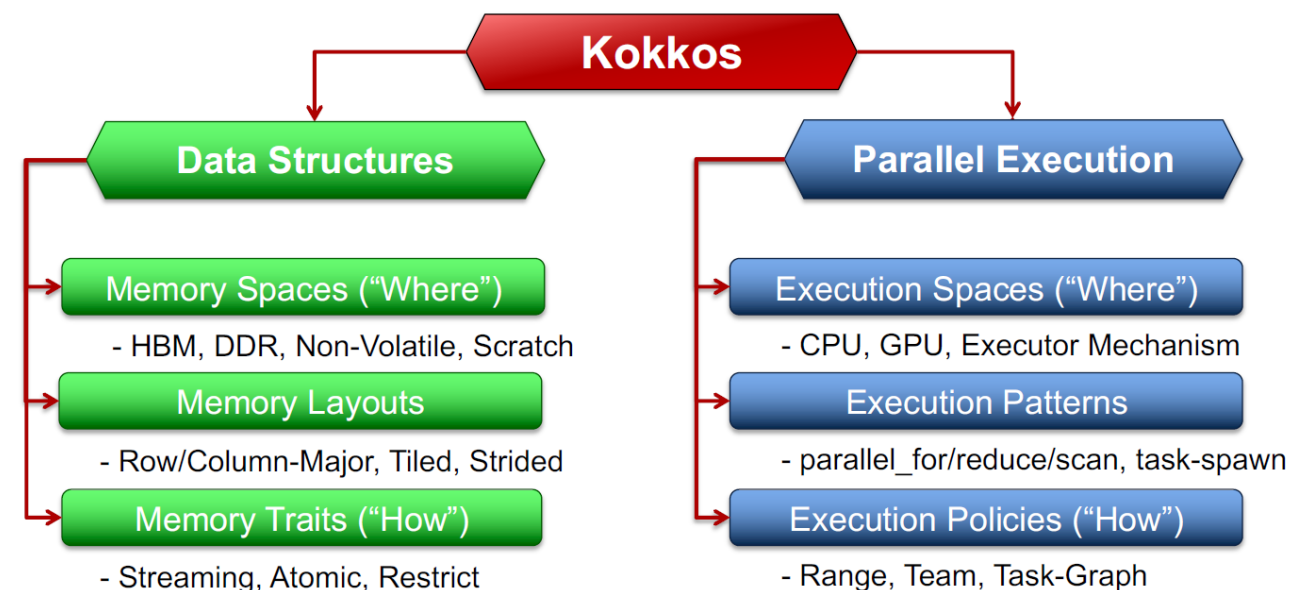
Currently supported backends and devices (03.2024):

- OpenMP, CUDA, HIP
- Intel, Nvidia, ARM, IBM, AMD

<https://github.com/kokkos/kokkos>



Kokkos Core Abstractions



Kokkos: typical computational kernel

Serial C/C++ (single CPU execution space)

```
inline void computeNewState(State &state , const Domain &dom, const SourceSinkData &ss) {  
  for (int j=0; j<dom.ny; j++) {  
    for (int i=0; i<dom.nx; i++) {  
      int ii = dom.getHaloExtension(i,j,dom.nx);  
  
      real z=state.z(ii);  
      real hold=state.h(ii);  
      real huold=state.hu(ii);  
      real hvold=state.hv(ii);  
      bool nodata=state.isnodata(ii);  
  
      // lots of computationally intensive code  
    }  
  }  
}
```

Kokkos: typical computational kernel

Serial C/C++ (single CPU execution space)

```
inline void computeNewState(State &state , const Domain &dom, const SourceSinkData &ss) {  
  for (int j=0; j<dom.ny; j++) {  
    for (int i=0; i<dom.nx; i++) {  
      int ii = dom.getHaloExtension(i,j,dom.nx);  
  
      real z=state.z(ii);  
      real hold=state.h(ii);  
      real huold=state.hu(ii);  
      real hvold=state.hv(ii);  
      bool nodata=state.isnodata(ii);  
  
      // lots of computationally intensive code  
    }  
  }  
}
```

```
inline void computeNewState(State &state , const Domain &dom, const SourceSinkData &ss) {  
  
  Kokkos::parallel_for( dom.nCellDomain , KOKKOS_LAMBDA (int iGlob) {  
    int ii = dom.getIndex(iGlob);  
  
    real z=state.z(ii);  
    real hold=state.h(ii);  
    real huold=state.hu(ii);  
    real hvold=state.hv(ii);  
    bool nodata=state.isnodata(ii);  
  
    // lots of computationally intensive code  
  })  
}
```

Kokkos C++

Kokkos: typical computational kernel

Serial C/C++ (single CPU execution space)

```
inline void computeNewState(State &state , const Domain &dom, const SourceSinkData &ss) {  
  for (int j=0; j<dom.ny; j++) {  
    for (int i=0; i<dom.nx; i++) {  
      int ii = dom.getHaloExtension(i,j,dom.nx);  
  
      real z=state.z(ii);  
      real hold=state.h(ii);  
      real huold=state.hu(ii);  
      real hvold=state.hv(ii);  
      bool nodata=state.isnodata(ii);  
  
      // lots of computationally intensive code  
    }  
  }  
}
```

This code works for single CPUs,
threaded CPUs (OpenMP) and GPUs
(CUDA)



```
inline void computeNewState(State &state , const Domain &dom, const SourceSinkData &ss) {  
  Kokkos::parallel_for( dom.nCellDomain , KOKKOS_LAMBDA (int iGlob) {  
    int ii = dom.getIndex(iGlob);  
  
    real z=state.z(ii);  
    real hold=state.h(ii);  
    real huold=state.hu(ii);  
    real hvold=state.hv(ii);  
    bool nodata=state.isnodata(ii);  
  
    // lots of computationally intensive code  
  })  
}
```

Implementing Kokkos is minimally
invasive (if parallelism is already well-
exposed).

Kokkos C++

Serial code is executed in host, parallel execution space determined at compilation or as a policy

Kokkos views

- Multidimensional array abstraction to simplify data management across diverse architectures

Defining all arrays as Kokkos Views

```
#include <Kokkos_Core.hpp>

typedef double      real;
typedef unsigned long  ulong;
typedef unsigned int  uint;

#ifdef __NVCC__
    typedef Kokkos::View<real*   , Kokkos::Device<Kokkos::Cuda, Kokkos::CudaUVMSpace>> realArr;
    typedef Kokkos::View<int*    , Kokkos::Device<Kokkos::Cuda, Kokkos::CudaUVMSpace>> intArr;
    typedef Kokkos::View<bool*   , Kokkos::Device<Kokkos::Cuda, Kokkos::CudaUVMSpace>> boolArr;
#else
    typedef Kokkos::View<real*   , Kokkos::LayoutRight> realArr;
    typedef Kokkos::View<int*    , Kokkos::LayoutRight> intArr;
    typedef Kokkos::View<bool*   , Kokkos::LayoutRight> boolArr;
#endif
```

Unified memory
visible for host and
device



No memory space is provided: the data resides
in the default memory space of the default
execution space (in this case host).

These are convenient general definitions, but can be fine-grained for each specific need

Kokkos vs. CUDA and OpenACC

- Avoid deep copies between CPU and GPU (CUDA, OpenMP/OpenACC)
- Avoid re-writing code: human mistakes, reproducibility, readability (CUDA, OpenMP/ OpenACC)
- Require target-specific adaptations of data structures to achieve good performance on both CPUs and GPUs (OpenMP/ OpenACC)
- Portability between different architectures (CUDA)
- Acceptable performance for both CPU and GPUs (CUDA, OpenMP/ OpenMP)

criterion	OpenMP/OpenACC (CPU+GPU)	CUDA (GPU)	Kokkos (CPU+GPU)
code clarity	high	low	medium
productivity	high/medium	low	medium
portability	low/medium	low	high
performance	high	high	high

The memory bottleneck: feeding the beast

GPU peak FLOPS grow $\sim 2\times$ every generation — but memory bandwidth grows $\sim 1.3\times$

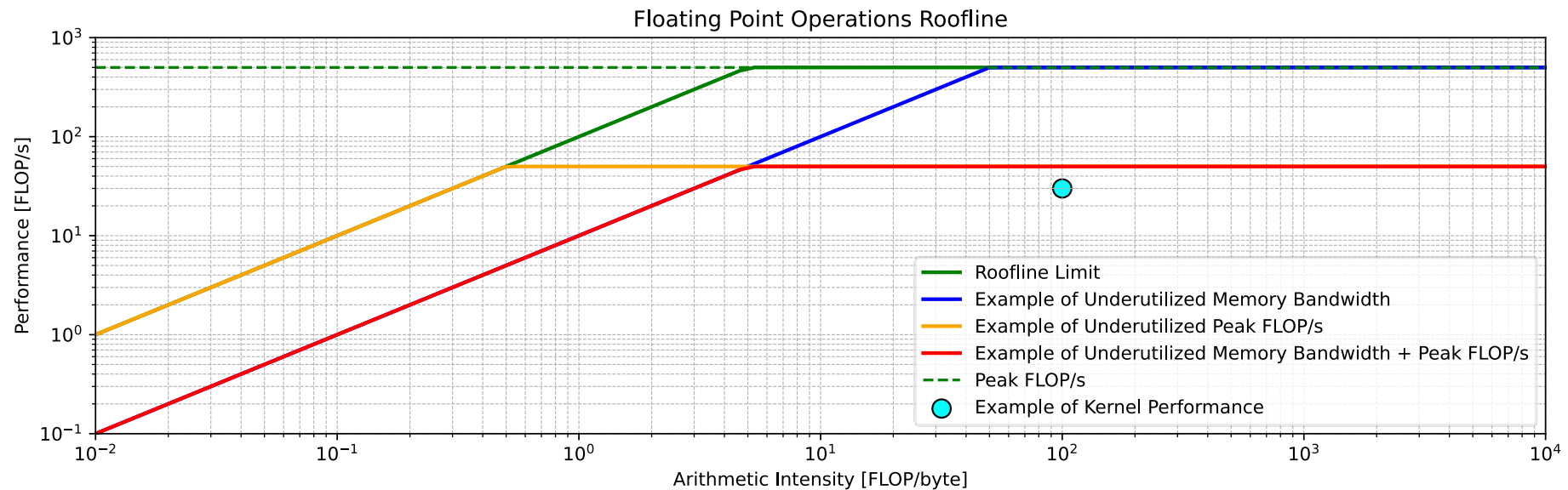
The roofline model: every workload has an arithmetic intensity threshold

Data transfer cost: cudaMemcpy / PCIe transfer can dominate execution time

On-node: HBM2e bandwidth $\sim 2\text{--}3\text{ TB/s}$ vs. DDR5 $\sim 80\text{ GB/s}$ ($\sim 30\times$ gap)

Mitigation strategies

- Overlap transfers with computation (async copies, streams)
- Keep data on device between kernels (minimize round-trips)
- Use unified memory with prefetch hints
- Fuse kernels to reduce intermediate transfers



On a GPU, it's not who computes the most that wins — it's who feeds the hardware most efficiently.

The abstraction spectrum

Low-level / Vendor-locked

High-level / Portable

CUDA / HIP

Maximum control
Vendor-locked

SYCL

Standard C++
Performance-portable

OpenMP / OpenACC

Directive-based
Easiest to adopt

Kokkos / RAJA

Max portability
DOE-backed

CUDA/HIP: deepest hardware access, but locks you to NVIDIA/AMD

SYCL: C++ expressiveness + cross-vendor GPU support

OpenMP/OpenACC: easiest to add to existing codebases

Kokkos/RAJA: best for long-lived HPC applications targeting multiple backends

Which model should you choose? A decision guide

Factor	CUDA	SYCL	OpenMP/OpenACC	Kokkos
Target hardware	NVIDIA only	Multi-vendor GPU/CPU/FPGA	CPU + GPU (compiler-dep.)	Multi-vendor GPU + CPU
Performance ceiling	Highest	High (near-CUDA)	Good (compiler-dep.)	High (backend-dep.)
Code longevity	Low (vendor-locked)	High (Khronos std)	High (ISO track)	High (DOE-backed)
Learning curve	Steep	Moderate	Low (directives)	Moderate
Integration	New project / CUDA-native	C++ projects, migration	Add #pragma to existing	C++ perf-portable apps
Ecosystem	Largest GPU ecosystem	Growing (Intel, OSS)	Broad (HPC standard)	Strong in DOE/exascale

Quick Decision Guide:

- NVIDIA-only, max performance? → CUDA
- Multi-vendor GPU, modern C++? → SYCL
- Quick GPU port of existing code? → OpenMP Target
- Long-lived portable C++ application? → Kokkos

No single model wins everywhere — choose based on your hardware target, code lifespan, and performance needs.

<https://forms.gle/jyddMK8x5Nbmx1g2A>



GPU programming models and languages

GPU programming in computational engineering

Escuela de Doctorado 2025/26

University of Zaragoza

Sergio Martínez-Aranda

sermar@unizar.es

Mario Morales-Hernández

mmorales@unizar.es

Fluid Dynamics Technologies - I3A, University of Zaragoza



Universidad
Zaragoza



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza



Instituto Universitario de Investigación
en Ingeniería de Aragón
Universidad Zaragoza